



UNIVERSIDAD TÉCNICA DE MANABÍ

UTM

Facultad De Ciencias Informatica

TECNOLOGIA DE LA INFORMACION

TEMA:

Actividad #1– U1: sistema de gestión de incidentes (HEP
DESK)

NOMBRE:

Chóez González Darwin Diodoro

DOCENTE:

Ing. Jorge Arun Zambrano cedeño, Mg.

MATERIA:

Desarrollo De Sistemas Informaticos

ABRIL – AGOSTO 2026

INTRODUCCIÓN

Actualmente los centros de datos requieren una gestión eficiente de incidentes técnicos relacionados con fallos de infraestructura, software, conectividad y hardware. La administración manual de incidentes genera retrasos, pérdida de trazabilidad y baja capacidad de respuesta.

Un sistema Help Desk permite centralizar incidentes, automatizar asignaciones, monitorear estados y generar notificaciones automáticas, mejorando la disponibilidad operativa del Data Center.

En esta actividad se diseña la arquitectura lógica de un sistema Help Desk aplicando UML, principios SOLID, Gitflow y patrones de diseño.

OBJETIVO

Diseñar la arquitectura de un sistema de software aplicando patrones de diseño y flujos de trabajo profesionales, fortaleciendo la capacidad de análisis y las buenas prácticas de ingeniería de software para soluciones escalables y mantenibles.

Diseñar la arquitectura lógica de un sistema de gestión de incidentes técnicos orientado a infraestructura de servidores y Data Center, aplicando patrones de diseño de software y flujos Gitflow para garantizar escalabilidad, mantenibilidad y reutilización.

ANÁLISIS DEL PROBLEMA

En un Data Center se presentan incidentes como:

- caída de servidores
- pérdida de conectividad
- errores de software
- fallos de hardware
- saturación de red

también pueden subsistir incidentes como:

- ◆ registro manual de incidencias
- ◆ asignación desorganizada
- ◆ ausencia de historial
- ◆ falta de alertas automáticas
- ◆ mala trazabilidad

para poder solucionar estos incidentes se requiere un sistema centralizado que permita:

- registrar incidentes
- clasificarlos
- asignar especialistas
- monitorear resolución

ACTORES PRINCIPALES

Usuario Final

Es la persona que detecta y reporta los incidentes y problemas técnicos mediante tickets

Funciones asignadas de responsabilidades

- Crear ticket
- Consultar estado
- Adjuntar evidencia
- Cerrar ticket
- Recibir notificaciones

Técnico de Soporte

Es el especialista responsable y encargado de resolver incidentes.

Funciones asignadas de Responsabilidades

- Revisar tickets asignados
- Diagnosticar fallos
- Actualizar estados
- Resolver incidents
- Agregar comentarios tecnicos

Especialidades:

- Red
- Hardware
- Software

Administrador de Sistemas

Es el encargado de gestionar recursos técnicos, realizar asignaciones y supervisar

Funciones de Responsabilidades

- Asignar técnicos
- Supervisar tickets
- Configurar prioridades
- Gestionar usuarios
- Generar reportes

REQUERIMIENTOS

Requerimientos Funcionales

- Registrar usuarios.
- Iniciar sesión.
- Crear tickets.
- Editar tickets.
- Consultar estado.

- Asignar técnico.
- Cambiar estado del ticket.
- Agregar comentarios.
- Notificar cambios.
- Cerrar tickets.
- Generar reportes.

Requerimientos No Funcionales

- Seguridad mediante autenticación.
- Disponibilidad 24/7.
- Interfaz amigable.
- Escalabilidad.
- Tiempo de respuesta menor a 3 segundos.
- Base de datos segura.
- Compatibilidad web.

Justificación del uso de los patrones de diseño de software (Singleton.Factory Method, Observer)

Justificación de la elección y uso técnico de los patrones de diseño

1. Patrón Singleton

El patrón Singleton garantiza que una clase tenga una única instancia global y proporciona un punto de acceso controlado a dicha instancia.

Aplicación en el sistema Help Desk

Se implementa en la clase TicketManager, encargada de administrar el registro, almacenamiento y seguimiento de tickets del sistema.

Problema que resuelve

En un sistema de gestión de incidentes, múltiples instancias del gestor de tickets podrían provocar:

- duplicidad de registros,
- inconsistencias de información,
- conflictos de sincronización,
- pérdida de control centralizado.

Singleton evita estos problemas al asegurar una sola instancia del motor de gestión.

El uso técnico permite:

- centralizar operaciones CRUD de tickets,
- mantener consistencia en estados,
- controlar acceso concurrente a datos,

- optimizar recursos del sistema.

2. Patrón Observer

El patrón Observer establece una relación uno-a-muchos entre objetos, permitiendo notificar automáticamente cambios de estado a los suscriptores.

Aplicación en el sistema Help Desk

Se utiliza en el módulo Notificador, encargado de alertar técnicos cuando:

- se crea un ticket,
- cambia prioridad,
- cambia estado,
- se asigna un nuevo incidente.

Problema que resuelve

- Sin Observer sería necesario programar notificaciones manuales mediante múltiples llamadas directas, generando:
- alto acoplamiento,
- código repetitivo,
- difícil mantenimiento.
- Observer desacopla el emisor del receptor.

Uso técnico

- Cuando se crea un ticket:
- El sistema genera incidente.
- Notificador detecta evento.
- Técnicos suscritos reciben alerta automática.

Es especialmente útil en sistemas de monitoreo y Data Center donde los eventos requieren respuesta inmediata.

3. Patrón Factory Method

El patrón Factory Method define una interfaz para crear objetos, delegando a subclases o métodos especializados la creación concreta.

Aplicación en el sistema Help Desk

Se implementa mediante la clase TicketFactory para crear diferentes tipos de incidentes:

- Hardware
- Red
- Software

Problema que resuelve

Sin Factory Method sería necesario usar múltiples condicionales lo cual produce:

- código rígido,
- baja extensibilidad,
- violación del principio Open/Closed.

Uso técnico

Factory centraliza la creación, según el tipo solicitado, devuelve el objeto correspondiente.

Beneficios técnicos

- encapsulación de creación,
- reducción de condicionales,
- código limpio,
- extensibilidad futura.
- Escalabilidad

Permite agregar nuevas categorías como: Seguridad, Base de datos y Virtualización sin modificar código existente.

JUSTIFICACIÓN DEL USO DE GITFLOW

Gitflow permite:

- control de versiones
- trabajo organizado
- integración segura
- historial claro

Beneficios:

- separación entre producción y desarrollo
- control de features
- rollback seguro

DIAGRAMA DE CLASE

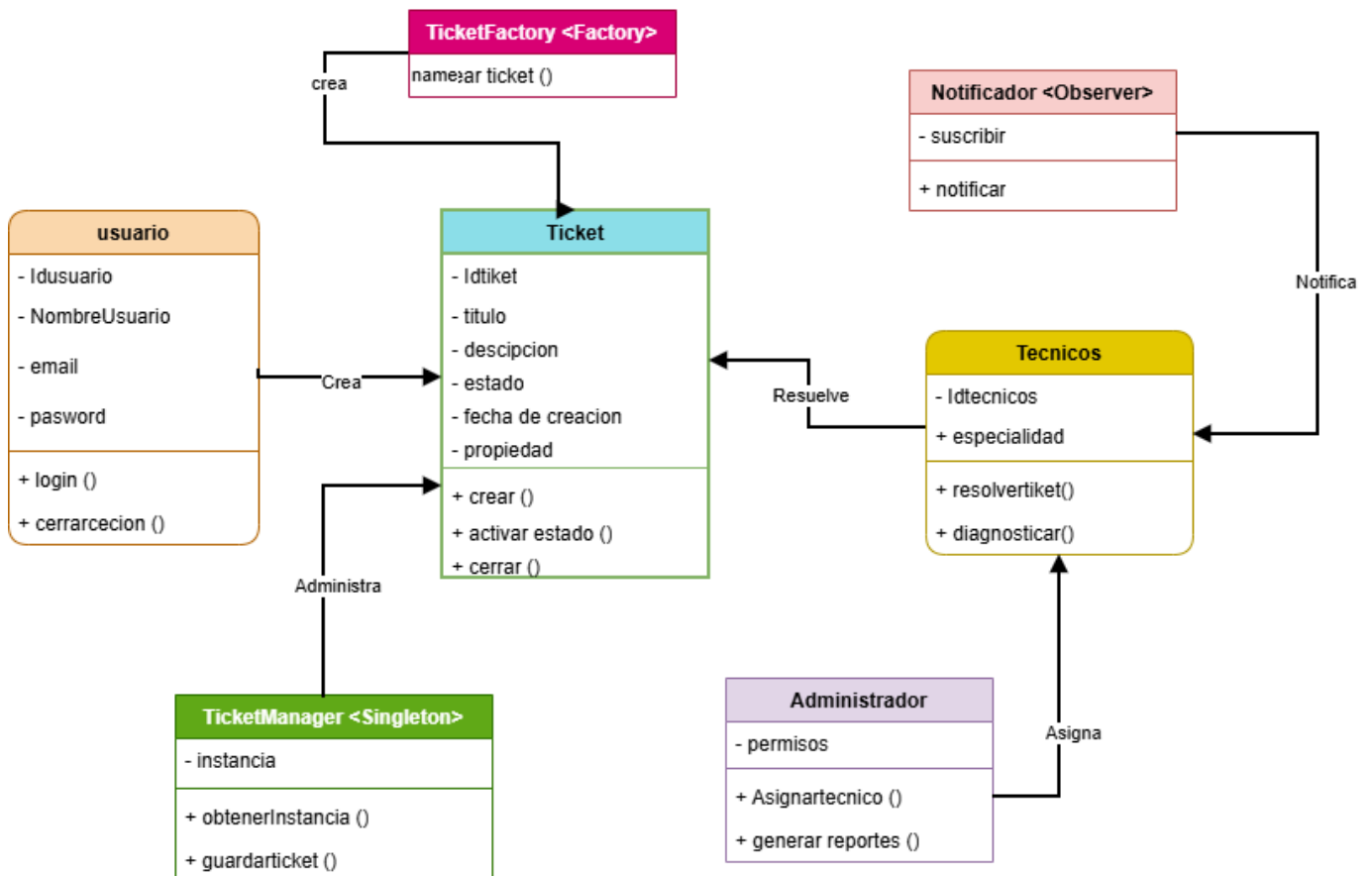


DIAGRAMA DE CASO DE USO

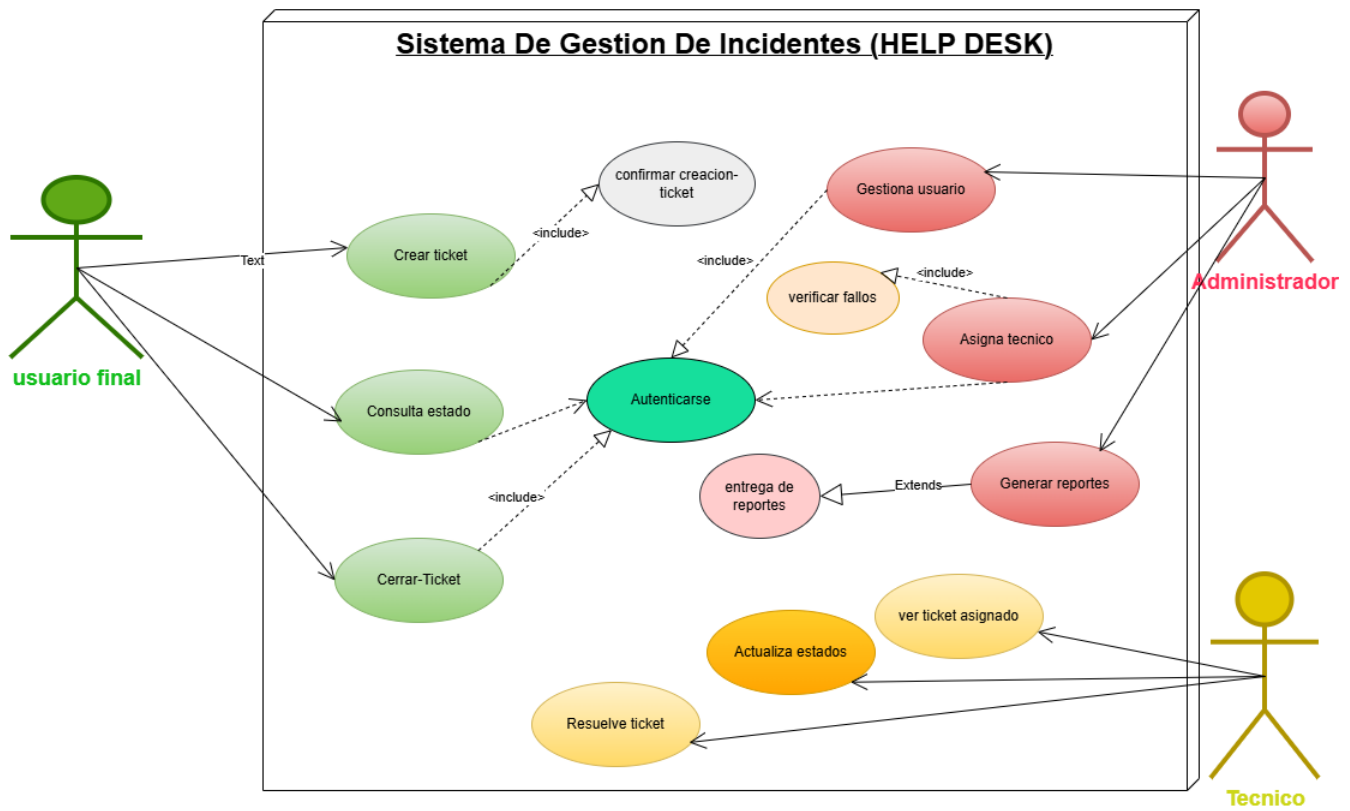
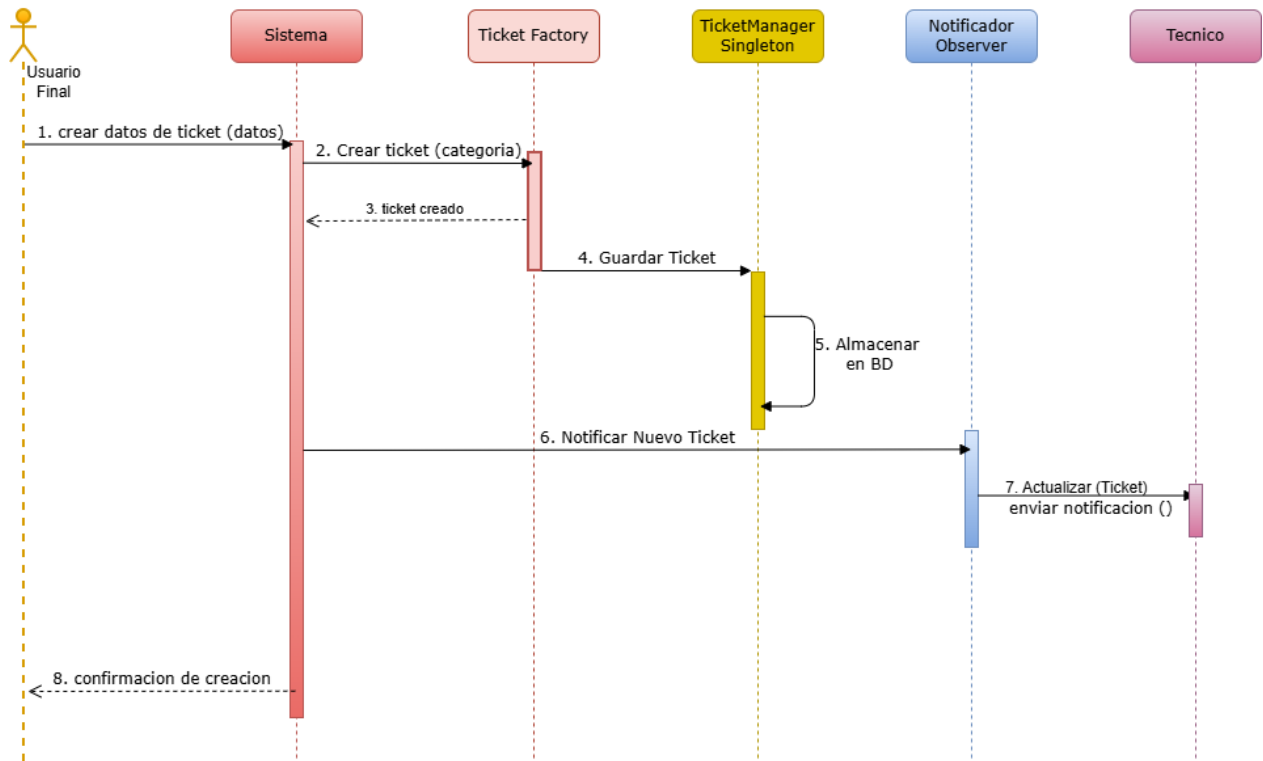
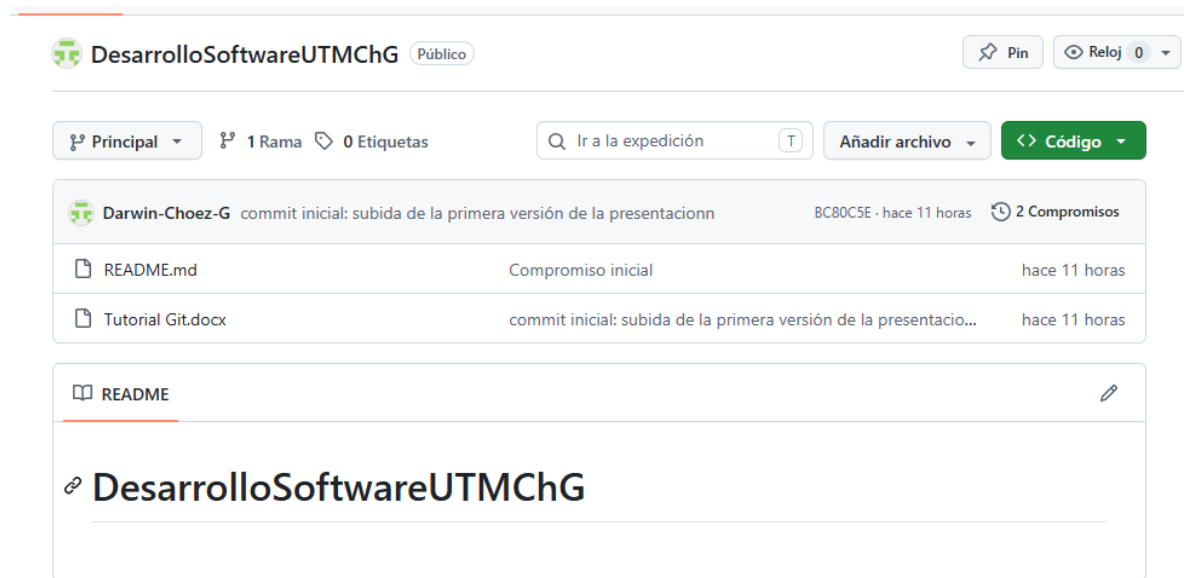


DIAGRAMA DE SECUENCIA

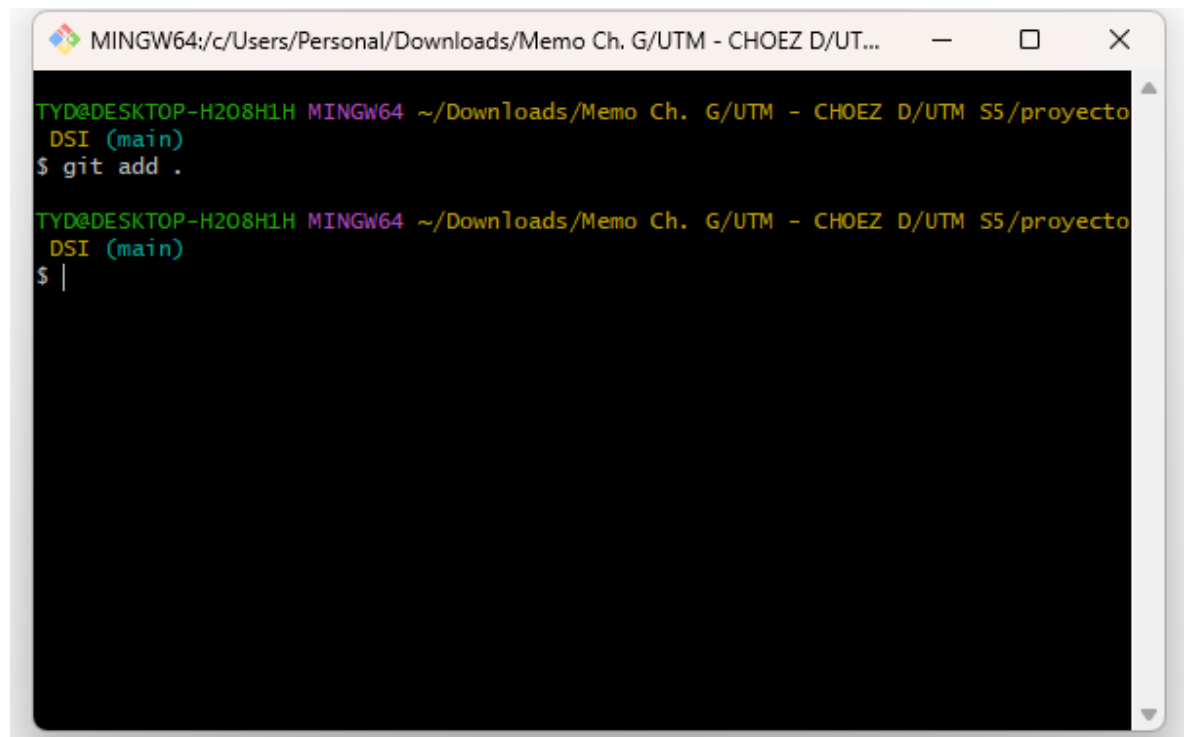


CAPTURAS DE PANTALLA

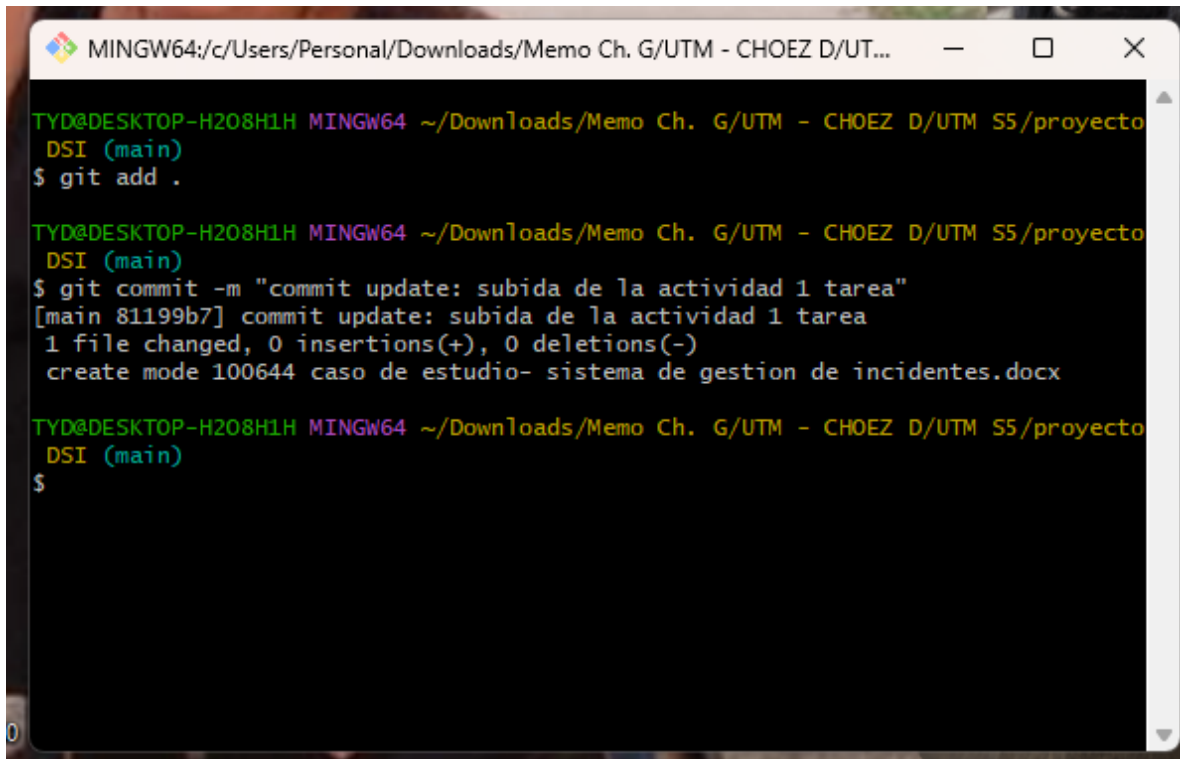
Capturas de pantalla del archivo subido a la nube por medio de comando



```
git add .
```




```
git commit -m "commit update: subida de la actividad 1 tarea"
```

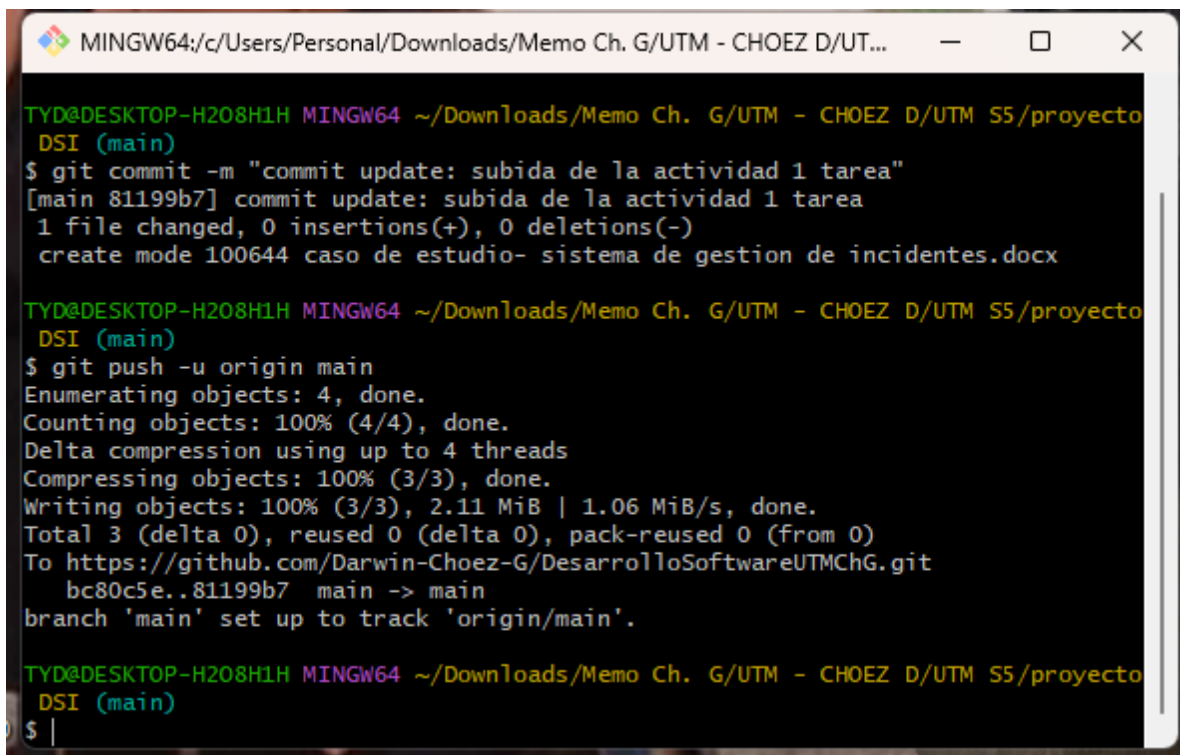


```
MINGW64; c:/Users/Personal/Downloads/Memo Ch. G/UTM - CHOEZ D/UT...
TYD@DESKTOP-H208H1H MINGW64 ~/Downloads/Memo Ch. G/UTM - CHOEZ D/UTM S5/proyecto
DSI (main)
$ git add .

TYD@DESKTOP-H208H1H MINGW64 ~/Downloads/Memo Ch. G/UTM - CHOEZ D/UTM S5/proyecto
DSI (main)
$ git commit -m "commit update: subida de la actividad 1 tarea"
[main 81199b7] commit update: subida de la actividad 1 tarea
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 caso de estudio- sistema de gestion de incidentes.docx

TYD@DESKTOP-H208H1H MINGW64 ~/Downloads/Memo Ch. G/UTM - CHOEZ D/UTM S5/proyecto
DSI (main)
$
```

```
git push -u origin main
```



```
MINGW64; c:/Users/Personal/Downloads/Memo Ch. G/UTM - CHOEZ D/UT...
TYD@DESKTOP-H208H1H MINGW64 ~/Downloads/Memo Ch. G/UTM - CHOEZ D/UTM S5/proyecto
DSI (main)
$ git commit -m "commit update: subida de la actividad 1 tarea"
[main 81199b7] commit update: subida de la actividad 1 tarea
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 caso de estudio- sistema de gestion de incidentes.docx

TYD@DESKTOP-H208H1H MINGW64 ~/Downloads/Memo Ch. G/UTM - CHOEZ D/UTM S5/proyecto
DSI (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 2.11 MiB | 1.06 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Darwin-Choez-G/DesarrolloSoftwareUTMChG.git
   bc80c5e..81199b7  main -> main
branch 'main' set up to track 'origin/main'.

TYD@DESKTOP-H208H1H MINGW64 ~/Downloads/Memo Ch. G/UTM - CHOEZ D/UTM S5/proyecto
DSI (main)
$ |
```

Link

[DesarrolloSoftwareUTMChG/caso de estudio- sistema de gestion de incidentes.docx at main · Darwin-Choez-G/DesarrolloSoftwareUTMChG](#)